

Tutorial - Semana 5 - Encontro 1

Introdução

O Gameplay Framework da Semana 4 (`BP_GameMode` , `BP_GameState` , `BP_PlayerController` , `BP_GameInstance`) está completo. A partir de agora, o Vertical Slice passa a acumular objetos do mundo com os quais o jogador interage — portas, alavancas, baús. Todos eles compartilham o mesmo problema: o `InteractionComponent` de `BP_Player` precisa poder "chamar" qualquer um desses objetos sem conhecer, um a um, a classe concreta de cada um. Este encontro constrói a solução para esse problema — a Blueprint Interface `BPI_Interactable` — e a implementa em um primeiro Actor de teste, preparando a base para o Encontro 2.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisitar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Compreender a comunicação desacoplada entre sistemas como problema universal de arquitetura de jogos.
- Diferenciar Blueprint Interface (contrato de comunicação) de Event Dispatcher (notificação de evento).
- Implementar `BPI_Interactable` e conectá-la ao `InteractionComponent` de `BP_Player` .

Resultado Esperado ao Final da Semana

Um objeto interativo funcional no Vertical Slice, implementando `BPI_Interactable` e reagindo via Event Dispatcher (Encontro 2) a uma ação do jogador, com o Gameplay Framework da Semana 4 permanecendo intacto.

Pré-requisitos

- Gameplay Framework da Semana 4 (`BP_GameMode` , `BP_GameState` , `BP_PlayerController` , `BP_GameInstance`) funcional.
- `BP_Player` controlável por Enhanced Input, com `InteractionComponent` já existente (ou a ser criado neste encontro, caso ainda não exista).

Antes de começar

O que você deverá possuir antes desta semana

- O projeto da Semana 4, com o Gameplay Framework completo e atribuído ao nível de teste.
- `BP_Player` funcional, com Enhanced Input configurado desde a Semana 2.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset novo. Este encontro trabalha exclusivamente com Blueprints do próprio projeto.

Projeto esperado

O projeto da Semana 4, com ao menos um Actor candidato no nível de teste (uma porta, alavanca, baú ou objeto equivalente já modelado ou representado por uma mesh simples) para se tornar o objeto interativo do desafio do Encontro 2.

Parte 1 — Blueprint Interface: contrato sem conhecer a implementação

Objetivo

Compreender Blueprint Interfaces como mecanismo de comunicação desacoplada e criar a interface `BPI_Interactable`.

Conceito

Sempre que um sistema precisa agir sobre outro sem conhecer sua implementação específica, toda engine precisa de um mecanismo que garanta apenas que o alvo "responde a uma chamada esperada", sem exigir que o chamador saiba como. É o caso do `InteractionComponent` de `BP_Player`: ele precisa poder dizer "interaja" para uma porta, uma alavanca ou um baú — três Actors com lógicas internas completamente diferentes — sem ter um bloco de decisão por tipo de objeto. A Unreal resolve isso com **Blueprint Interfaces**: uma interface declara funções sem implementação, e qualquer Actor pode implementá-la. Quem chama a função via interface nunca precisa conhecer a classe concreta do alvo — apenas que ele implementa o contrato.

Passo a passo

1. No Content Browser, navegar até `Blueprints/Interactables/` (criar a subpasta caso ainda não exista, conforme `PROJECT_ARCHITECTURE.md`).
2. Clicar com o botão direito, escolher Blueprint Class e, na aba "All Classes"/seção específica, selecionar "Blueprint Interface".
3. Nomear a interface como `BPI_Interactable`, conforme a convenção de prefixo `BPI_` do `PROJECT_ARCHITECTURE.md`.
4. Abrir `BPI_Interactable` e, na aba Functions, criar uma função chamada `Interact`, sem corpo de execução (interfaces não implementam lógica).
5. Adicionar, se desejado, um parâmetro de entrada simples à função `Interact` (por exemplo, uma referência ao Actor que iniciou a interação), mantendo a assinatura mínima necessária.
6. Compilar e salvar a interface.

Resultado esperado

Uma interface `BPI_Interactable` criada em `Interactables/`, contendo apenas a declaração da função `Interact`, sem qualquer lógica implementada dentro dela.

Verificando

Abrir `BPI_Interactable` novamente e confirmar que a função `Interact` aparece na aba Functions com o ícone de interface (sem grafo de execução editável dentro da própria interface).

Problemas comuns

- **Tentar adicionar lógica dentro da interface:** interfaces não implementam comportamento; se o Event Graph pedir lógica, o Blueprint criado é uma classe comum, não uma interface — revisar o

tipo escolhido na criação.

- **Esquecer o prefixo `BPI_`**: sempre nomear como `BPI_Interactable`, nunca `Interactable` ou `BP_Interactable`, conforme `PROJECT_ARCHITECTURE.md`.

Boas práticas

Mantenha a interface com o menor número possível de funções — apenas o que realmente precisa ser chamado de forma genérica por outro sistema. Funções auxiliares específicas de um Actor pertencem ao próprio Actor, não à interface.

Comparação com Unity

Interfaces em C# resolvem o mesmo problema de contrato sem herança de implementação: definir o que um objeto deve responder, não como. A diferença está na camada de execução — Blueprint Interfaces podem ser chamadas mesmo quando o chamador não tem referência de classe concreta ao alvo (Call Function on Interface), enquanto na Unity o mesmo padrão tipicamente exige `GetComponent<IInteragivel>()` para obter a referência tipada pela interface, em C#.

Parte 2 — Implementando a interface em um Actor

Objetivo

Implementar `BPI_Interactable` em um Actor concreto do nível de teste e preparar o `InteractionComponent` de `BP_Player` para chamá-la de forma genérica.

Conceito

Uma interface só se torna útil quando implementada por uma classe concreta. Implementar uma interface significa apenas prover a função exigida pelo contrato — não herdar comportamento nem duplicar estado. Um Actor pode implementar quantas interfaces forem necessárias, mantendo cada uma delas isolada de sua lógica interna específica. É essa implementação que permite ao `InteractionComponent` chamar `Interact` em qualquer Actor próximo, sem saber se é uma porta, uma alavanca ou qualquer outro objeto futuro do Vertical Slice.

Passo a passo

1. Escolher um Actor já existente no nível de teste (ou criar um novo Blueprint em `Interactables/`, herdando de Actor, caso nenhum candidato exista ainda).
2. Abrir o Actor escolhido e, em Class Settings, adicionar `BPI_Interactable` na lista de Implemented Interfaces.
3. Compilar o Actor; a função `Interact` deve aparecer disponível para implementação na aba Functions, sob a seção Interfaces.
4. Implementar `Interact` com uma lógica simples e visível (por exemplo, um Print String ou uma mudança temporária de cor do material), apenas para confirmar que a chamada está funcionando — a reação definitiva via Event Dispatcher será construída no Encontro 2.
5. No `InteractionComponent` de `BP_Player` (ou no Actor que o hospeda), localizar o ponto em que a interação é disparada (por exemplo, ao pressionar a tecla de interação com um objeto detectado por overlap ou trace) e adicionar o nó "Call Function on Interface" (ou equivalente), chamando `Interact` sobre a referência do Actor detectado.
6. Compilar e salvar todos os Blueprints alterados.
7. Testar em modo Play: aproximar-se do Actor implementado, acionar a interação pelo `InteractionComponent`, e confirmar que a lógica simples de `Interact` é executada.

Resultado esperado

Um Actor do nível de teste implementando `BPI_Interactable`, com a função `Interact` executando uma reação simples e visível quando chamada pelo `InteractionComponent` de `BP_Player`, sem que o `InteractionComponent` conheça a classe concreta do Actor.

Verificando

Ao acionar a interação próximo ao Actor, a reação simples implementada em `Interact` deve ocorrer (Print String, mudança de cor ou equivalente). Se possível, testar com um segundo Actor diferente também implementando a interface, confirmando que o mesmo `InteractionComponent` funciona para ambos sem alteração de código.

Problemas comuns

- **Confundir implementar interface com herdar classe:** implementar `BPI_Interactable` não copia nenhum comportamento pronto; a lógica de `Interact` precisa ser escrita em cada Actor que a implementa.
- **Chamar a função diretamente pela referência de classe, em vez de via interface:** isso reintroduz o acoplamento que a interface deveria eliminar; sempre usar "Call Function on

Interface" a partir do `InteractionComponent` .

- **`InteractionComponent` não detectar o Actor:** revisar o mecanismo de detecção (overlap, trace ou equivalente) usado pelo Component antes de suspeitar da interface.

Boas práticas

Ao testar a chamada via interface, use sempre uma reação temporária e óbvia (Print String, cor) antes de implementar a reação definitiva — isso isola erros de detecção/chamada de erros da reação em si, o que facilita a depuração no Encontro 2.

Comparação com Unity

O equivalente, em Unity, é obter a referência tipada pela interface via

`GetComponent<IInteragivel>()` a partir do script que detecta o objeto (por exemplo, dentro de um `OnTriggerEnter` ou de um raycast), chamando o método da interface sobre essa referência. O princípio é idêntico: quem chama nunca precisa do tipo concreto, apenas do tipo de interface.

Desafio

Não há desafio de liberdade de solução neste encontro — a criação e implementação da interface é demonstração e adaptação guiada, preparando a base técnica para o desafio do Encontro 2 (objeto interativo com Event Dispatcher e mecanismo de acionamento livre).

Ao final da semana

Este resultado será consolidado ao final do Encontro 2. Ao final deste Encontro 1, o Vertical Slice deve possuir `BPI_Interactable` criada em `Interactables/` e implementada em ao menos um Actor do nível de teste, com o `InteractionComponent` de `BP_Player` já chamando essa interface de forma genérica — a base de comunicação desacoplada descrita na seção 7 do `PROJECT_ARCHITECTURE.md`.

Checklist

☐ `BPI_Interactable` criada em `Blueprints/Interactables/`, com a função `Interact` declarada

- ☐ Ao menos um Actor do nível de teste implementando `BPI_Interactable`, com `Interact` executando uma reação simples
- ☐ `InteractionComponent` de `BP_Player` chamando `Interact` via "Call Function on Interface"
- ☐ Teste em modo Play confirmando a reação simples ao acionar a interação
- ☐ Nomenclatura e localização conforme `PROJECT_ARCHITECTURE.md` (`BPI_` e subpasta `Interactables/`)

Glossário

- **Blueprint Interface:** classe especial da Unreal que declara funções sem implementação, permitindo que Actors distintos as implementem e sejam chamados de forma genérica, sem exposição da classe concreta.
- **Call Function on Interface:** nó de Blueprint usado para chamar uma função de interface sobre uma referência de Actor, sem exigir conversão para a classe concreta.
- **Contrato de comunicação:** conceito universal segundo o qual um sistema define apenas o que deve existir em outro sistema, nunca como esse comportamento é implementado.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Blueprints Visual Scripting in Unreal Engine. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprints-visual-scripting-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução a Blueprint Interfaces. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — Interfaces em C#, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de Blueprint Interfaces; **Mathew Wadstein**, explicação pontual de WTF Is? Interface.